

Introduction to Data Science

Working with the command line

Simon Munzert

Hertie School | GRAD-C11/E1339

Table of contents

1. Introduction¹
2. Shell basics
3. Help!
4. Navigating your system
5. Managing your files
6. Working with text files
7. Redirects, pipes, and loops
8. Scripting
9. The shell in the age of AI agents

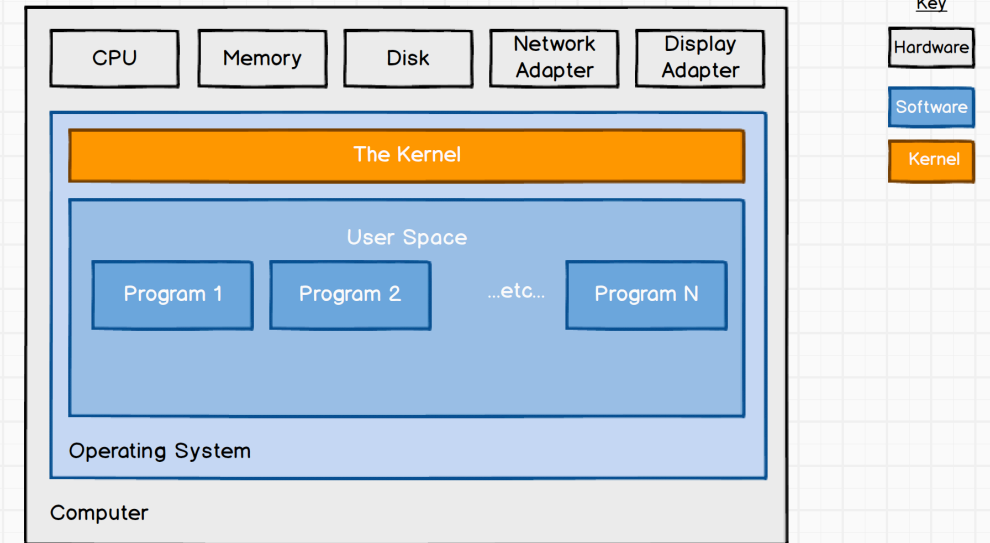
¹ Much of this lecture draws on materials from Grant McDermott's excellent *Data Science for Economists* class.

Introduction

What is the command line? What is the shell?

A computer in a nutshell

- The **operating system (OS)** is system software that interfaces with (and manages access to) a computer's hardware. It also provides software resources.
- The OS is divided into the kernel and user space.
- The **kernel** is the core of the OS. It's responsible for interfacing with hardware (drivers), managing resources etc. Running software in the kernel is *extremely* sensitive! That's why users are kept away from it.
- The **user space** provides an interface for users, who can run programs/applications on the machine. Hardware access of programs (e.g., memory usage) is managed by the kernel. Programs in user space are essentially in sandboxes, which sets a limit to how much damage they can do.

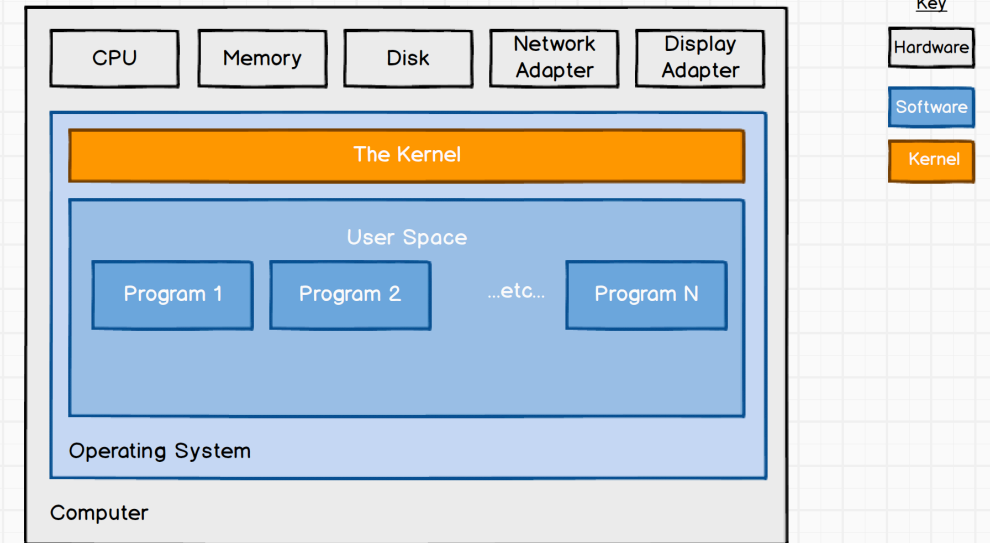


Credit Dave Kerr

What is the command line? What is the shell? (cont.)

A computer in a nutshell

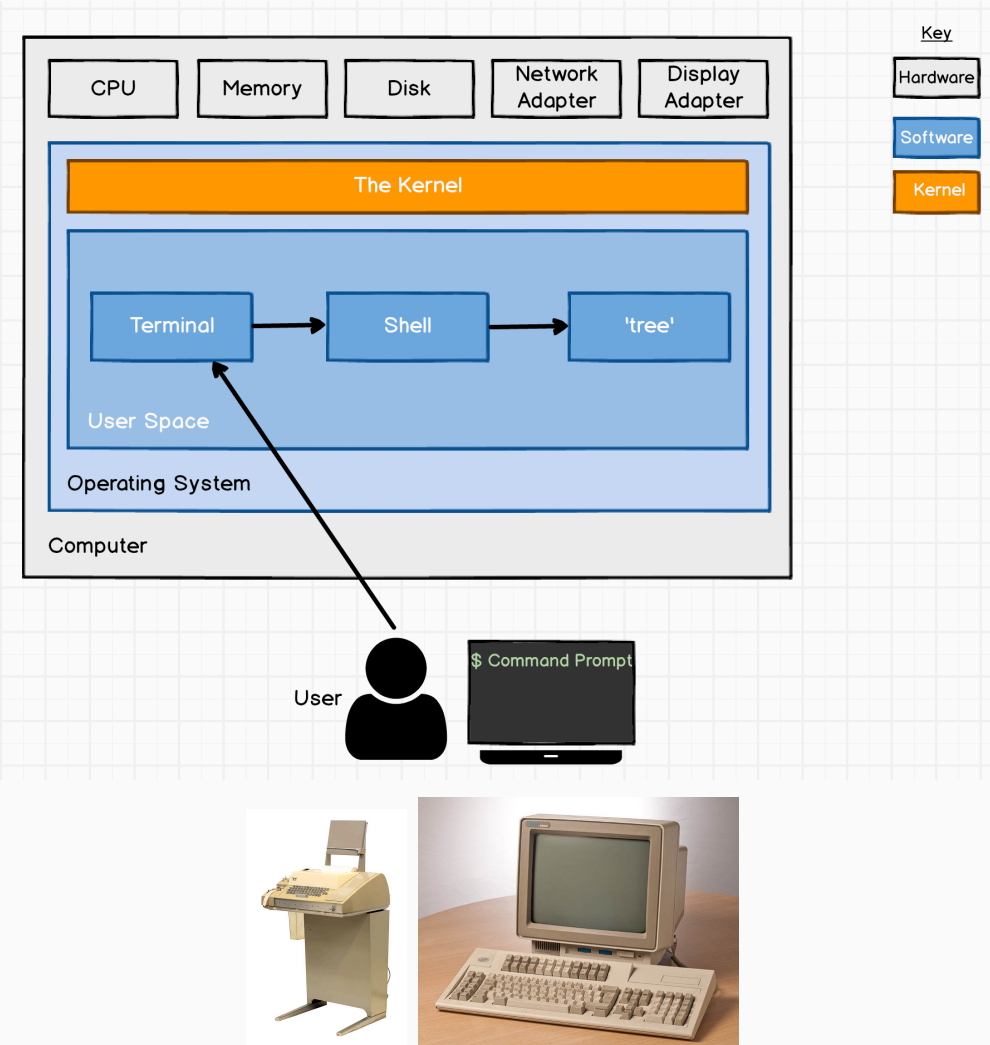
- The **shell** is just a general name for any user space program that allows access to resources in the system, via some kind of interface.
- Shells come in many different flavours but are generally provided to aid a human operator in accessing the system. This could be interactively, by **typing at a terminal**, or via **scripts**, which are files that contain a sequence of commands.
- Modern computers use **graphical user interfaces (GUIs)** as the standard tool for human-computer interaction.
- **Why "kernel" and "shell"?** The kernel is the soft, edible part of a nut or seed, which is surrounded by a shell to protect it. Useful metaphor, no?



What is the command line? What is the shell? (cont.)

Interacting with the shell

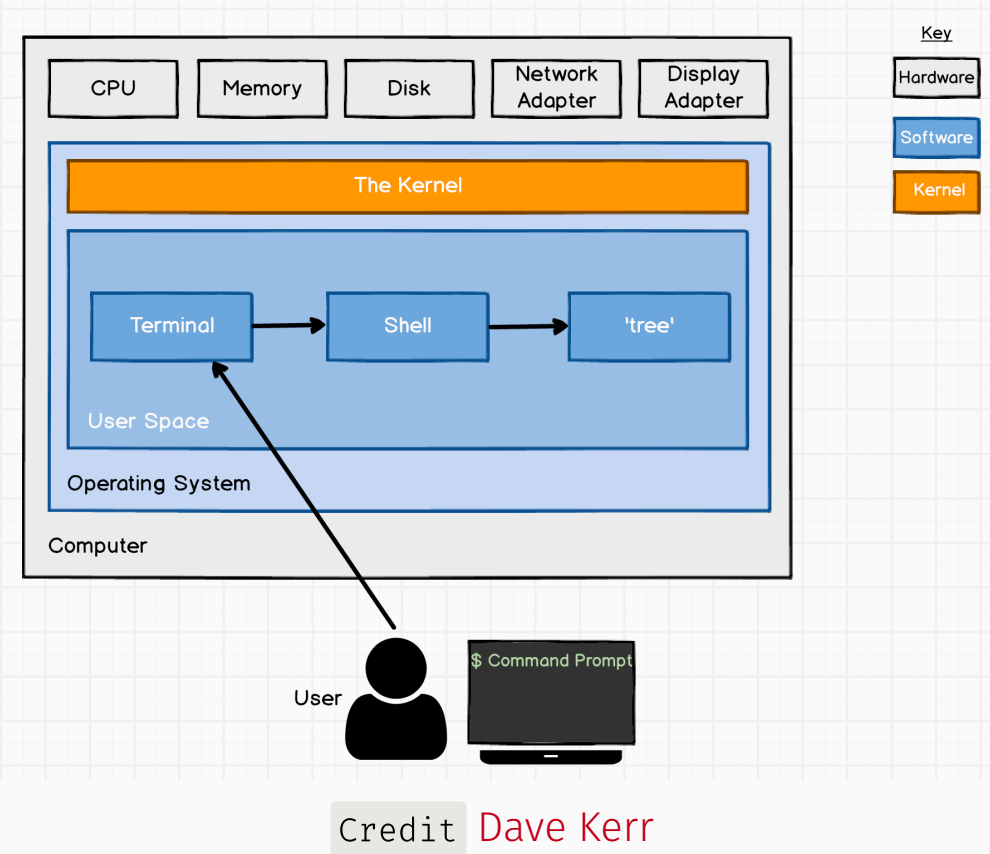
- Things are still a bit more complicated.
- We're not directly interacting with the "shell" but using a **terminal**.
- A terminal is just a program that **reads input** from the keyboard, **passes that input** to another program, and **displays the results** on the screen.
- A shell program on its own does not do this - it requires a terminal as an interface.
- **Why "terminal"?** Back in the old days (even before computer screen existed), terminal machines (hardware!) were used to let humans interface with large machines ("mainframes"). Often many terminals were connected to a single machine.
- When you want to work with a computer in a data center (or remotely ~ cloud computing), you'll still do pretty much the same.



What is the command line? What is the shell? (cont.)

Interacting with the shell

- Terminals are really quite simple - they're just interfaces.
- The first thing that a terminal program will do is run a shell program - a program that we can use to operate the computer.
- **Back to the shell:** the shell usually takes input (a) interactively from the user via the terminal's **command line**. (b) executes scripts (without command line).
- In interactive mode the shell then returns output (a) to the terminal where it is printed/shown. (b) to files or other locations.
- The command line (or command prompt) represents what is shown and entered in the terminal. They can be **customized** (e.g., with color highlighting) to make interaction more convenient.



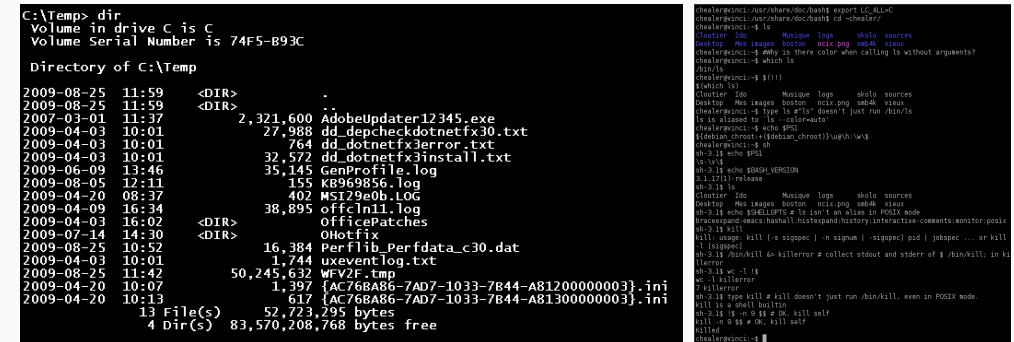
What is the command line? What is the shell? (cont.)

Shell variants

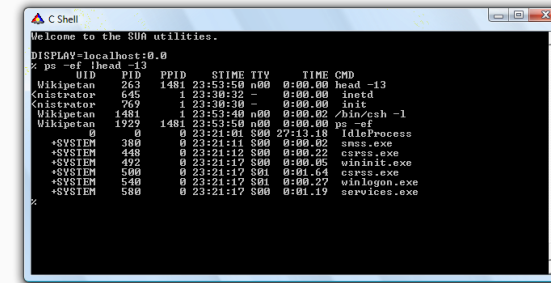
- It is important to note that there are many different shell programs, and they **differ in terms of functionality**.
- On most Unix-like systems, the default shell is a program called **bash**, which stands for "Bourne Again Shell".
- Other examples are the **Z Shell** (or zsh; default on MacOS)¹, **PowerShell** (the default shell on modern Windows), the legacy **Windows Command Prompt** (cmd.exe), **C Shell**, and many more.
- When a terminal opens, it will immediately start the user's preferred shell program. (This can be **changed**.)

¹That's what I use, which is why the shell on the following slides might look a bit different. Also, some commands/programs might or might not be available (or need to be installed).

Left: Command Prompt, right: Bash



Left: C Shell, right: more shells

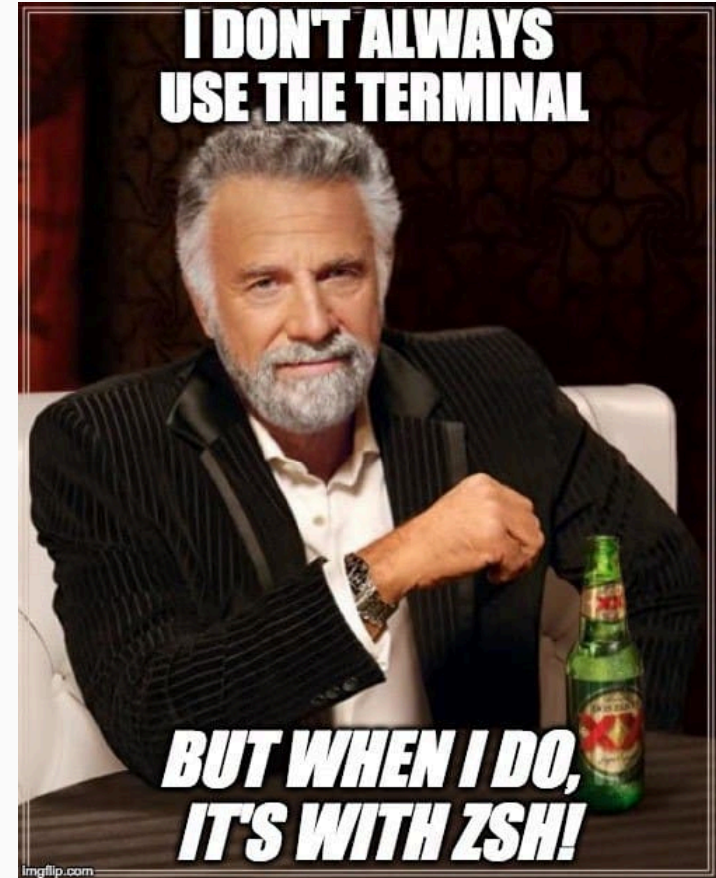


Credit **Read-back spider/Dave Kerr**

What is the command line? What is the shell? (cont.)

Summing things up

- All that being said, don't be thrown off by terminology: *shell*, *terminal*, *command prompt*, *command line*, *bash*, etc.
- In everyday usage they are all referring to a **command line interface** (CLI) with which we can talk to our computer, execute (chains of) programs, wrangling input and output, and so much more.



Why bother with the shell?

Why using this...

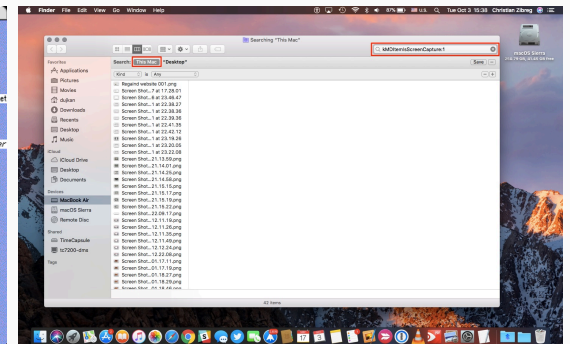
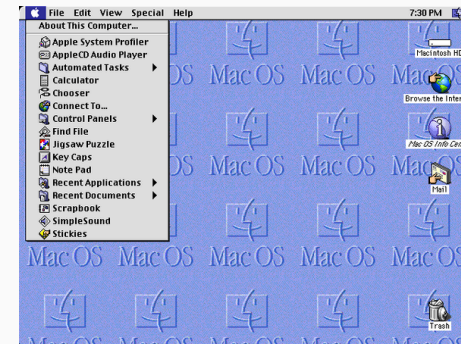
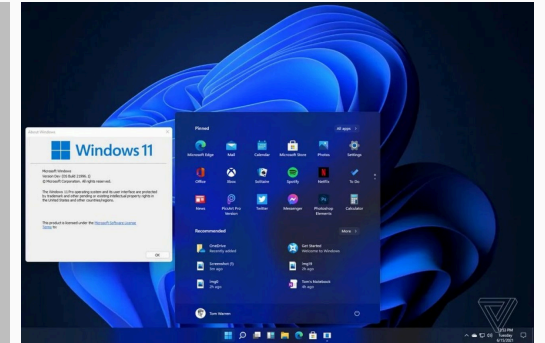
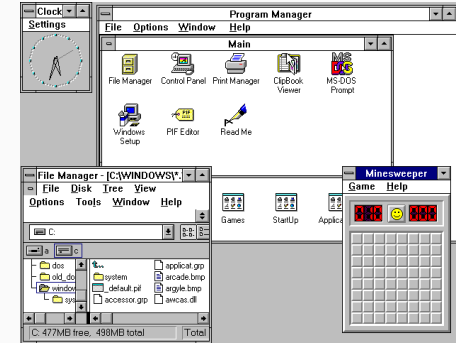
```
kg@KW ~ /jterm-test
→ iterm-test git:(master) X ls -lah
total 0
drwxr-xr-x  11 kg  staff   352B Aug 13 13:04 .
drwxr-xr-x@ 109 kg  staff   3.4K Aug 13 17:14 ..
drwxr-xr-x  12 kg  staff   384B Aug 13 17:14 .git
-rw-r--r--   1 kg  staff    0B Aug 13 11:50 .gitignore
-rw-r--r--   1 kg  staff    0B Aug 13 11:47 README.md
drwxr-xr-x   4 kg  staff   128B Aug 13 13:03 app
-rw-r--r--   1 kg  staff    0B Aug 13 11:50 docker-compose.yml
drwxr-xr-x   2 kg  staff   64B Aug 13 11:48 node_modules
-rw-r--r--   1 kg  staff    0B Aug 13 11:50 package-lock.json
-rw-r--r--   1 kg  staff    0B Aug 13 11:50 package.json
drwxr-xr-x   2 kg  staff   64B Aug 13 11:49 storage
→ iterm-test git:(master) X git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file> ..." to unstage)

    new file:   app/app.js

Untracked files:
  (use "git add <file> ..." to include in what will be committed)

    app/styles.scss
→ iterm-test git:(master) X
```

... instead of this?



Why bother with the shell? (cont.)

1. **Speed.** Typing is fast: A skilled shell user can manipulate a system at dazzling speeds just using a keyboard.
2. **Power.** Both for executing commands and for fixing problems. There are some things you just can't do in an IDE or GUI.
3. **Reproducibility.** Scripting is reproducible, while clicking is not.
4. **Portability.** A shell can be used to interface to almost any type of computer, from a mainframe to a Raspberry Pi, in a very similar way — and it is often the only game in town for high performance computing.
5. **Automation/Scripting.** Shells are programmable: Working in the shell allows you to program workflows, that is create scripts to automate time-consuming or repetitive processes.
6. **Become a marketable data scientist.** Modern programming is often polyglot, and modern solutions are often built to run in containers on Linux. The shell provides the common interface for all this tooling. In short, **the shell is having a renaissance** in the age of data science.
7. **Work with AI agents.** Coding agents like **Claude Code**, **Codex CLI**, or **Gemini CLI** act on your machine through shell commands — to supervise them, you must be able to read them. More on this **later**.

The Unix philosophy

The shell tools that we're going to be using today have their roots in the **Unix** family of operating systems originally developed at Bells Labs in the 1970s.

Besides paying homage, acknowledging the Unix lineage is important because these tools still embody the "**Unix philosophy**":



Do One Thing And Do It Well

By pairing and chaining well-designed individual components, we can build powerful and much more complex larger systems.

You can see why the Unix philosophy is also referred to as "minimalist and modular".

This 50 year-old design principle has aged remarkably well: Small, composable, text-based tools with predictable inputs and outputs turn out to be the ideal action space for LLM-based agents, which are good at text in, text out operations.

Things to use the shell for

- Version control with Git
- Renaming and moving files *en masse*
- Finding things on your computer
- Combining and manipulating PDFs
- Installing and updating software
- Scheduling tasks
- Monitoring system resources
- Connecting to cloud environments
- Running analyses ("jobs") on super computers
- Working with AI coding agents
- etc.

Shell basics

First look

Let's open up our shell.

A convenient way to do this is through **RStudio's built-in Terminal**.

- Hitting **Shift + Alt + T** (or **Shift + ⌘ + R** on a Mac) will cause a "Terminal" tab to open up next to the "Console" tab.
- Your system default shell is loaded. To find out what that is, type:

```
$ echo $SHELL
```

```
/bin/zsh
```

- Ok, it's Z bash in my case.

Of course, it's always possible to open up the Shell directly if you prefer.

- **Linux**
- **Mac**
- Windows: use **Git Bash** (installed together with Git) or a full Linux environment via **WSL** — the built-in PowerShell/cmd.exe won't understand most of the commands we cover today

First look (cont.)

You should see something like:

```
username@hostname:~`$`
```

This is shell-speak for: "Who am I and where am I?"

- `username` denotes a specific user (one of potentially many on this computer).
- `@hostname` denotes the name of the computer or server.
- `:~` denotes the directory path (where `~` signifies the user's home directory).
- `$` (or maybe `%`) denotes the start of the command prompt.

(For a special "superuser" called root, the dollar sign will change to a `#`).

Useful keyboard shortcuts

What you want to do	macOS	Windows (Git Bash) / Linux
Autocomplete commands and paths	Tab	Tab
Scroll through previous commands	↑ / ↓	↑ / ↓
Search your command history	Ctrl + r, then type	Ctrl + r, then type
Jump to beginning / end of line	Ctrl + a / Ctrl + e	Ctrl + a / Ctrl + e
Jump back / forward one word	⌘ + ← / ⌘ + →	Ctrl + ← / Ctrl + →
Delete everything left / right of cursor	Ctrl + u / Ctrl + k	Ctrl + u / Ctrl + k
Delete the word before the cursor	Ctrl + w	Ctrl + w
Copy / paste	⌘ + c / ⌘ + v	Ctrl + Shift + c / Ctrl + Shift + v
Clear the screen	Ctrl + l	Ctrl + l
Abort the running command	Ctrl + c	Ctrl + c

When a shortcut "doesn't work", it is usually being intercepted by the OS or by the app hosting your terminal (e.g., RStudio).

Syntax

All Bash commands have the same basic syntax:

command option(s) argument(s)

Examples:

```
$ ls -lh ~/Documents/
```

```
$ sort -u myfile.txt
```

Syntax (cont.)

All Bash commands have the same basic syntax:

command option(s) argument(s)

Examples:

```
$ `ls` -lh ~/Documents/
```

```
$ `sort` -u myfile.txt
```

Commands

- You don't always need options or arguments. (For example, `$ ls ~/Documents/` and `$ ls -lh` are both valid commands that will yield output.)
- However, you always need a command.

Syntax (cont.)

All Bash commands have the same basic syntax:

command **option(s)** **argument(s)**

Examples:

```
$ ls -lh ~/Documents/
```

```
$ sort -u myfile.txt
```

Options (also called **Flags**)

- Start with a dash, and are usually one letter.
- Multiple options can be chained together under a single dash.

```
$ ls -l -a -h /var/log ## This works
```

```
$ ls -lah /var/log ## So does this
```

- An exception is with (rarer) options requiring two dashes:

```
$ sort --reverse --unique myfile.txt ## same as:
```

- Think it's difficult to memorize what the individual letters stand for? You're totally right.
- Plus: *which* options exist depends on the program's flavor...

Syntax (cont.)

All Bash commands have the same basic syntax:

command option(s) argument(s)

Examples:

```
$ ls -lh `~/Documents/`
```

```
$ sort -u `myfile.txt`
```

Arguments

- Tell the command *what* to operate on.
- Totally depends on the command what legit inputs are.
- Can be a file, path, a set of files and folders, a string, and more
- Sometimes more than just one argument is needed:

```
$ mv pics/cat.JPG best-pics/cat.jpeg
```

Help!

Multiple ways to get help

Overview

- The `man` tool can be used to look at the manual page for a topic.
- The `man` pages are grouped into sections, we can see them with `man man`.
- The `tldr` tool shows a very short description of a tool, which covers the most common use cases only. Similar tools/services: `cheat` and `cheat.sh`.
- And, of course, **asking an AI assistant** has become the default way to get help. That's fine — but make it a habit to read and understand a command before you run it (more on this [later](#)).
- Actually, typing `man bash` and reading/skimming the whole thing might be a good start to learn basic command line speak.



Getting help with man

The `man` command ("manual pages") is your friend if you ever need help.

```
$ man ls
```

```
LS(1)                      General Commands Manual          LS(1)
```

NAME

```
ls - list directory contents
```

SYNOPSIS

```
ls [-@ABCFGHILOPRSTUwabcdefghijklmnpqrstuvwx1%,] [--color=when]
[-D format] [file ...]
```

DESCRIPTION

For each operand that names a file of a type other than directory, `ls` displays its name as well as any requested, associated information. For each operand that names a file of type directory, `ls` displays the names of files contained within that directory, as well as any requested, associated information.

If no operands are given, the contents of the current directory are displayed. If more than one operand is given, non-directory operands are displayed first; directory and non-directory operands are sorted separately and in lexicographical order.

Getting help with man (cont.)

Manual pages are shown in the shell *pager*. Here are the essentials to navigate through contents presented in the pager:

- `d` - Scroll down half a page
- `u` - Scroll up half a page
- `j` / `k` - Scroll down or up a line. You can also use the arrow keys for this
- `q` - Quit
- `/pattern` - Search for text provided as "pattern"
- `n` - When searching, find the next occurrence
- `N` - When searching, find the previous occurrence

These and other `man` tricks are detailed in the help pages (hit "h" when you're in the pager for an overview).



RTFM

Navigating your file system

Navigating your file system

- We're all so used to a graphical user interface that switching to the shell can take some time to get used to.
- Modern operating systems increasingly abstract away from underlying file systems (think about iOS, Android).
- For data science operations it is key that you're able to efficiently navigate your system to get information on files and folders.
- Some questions that will pop up:
 - What is my home directory?
 - In which directory am I currently operating?
 - Where is my stuff?
 - Where do I want to put my stuff?
 - How do I navigate from here to there?



Navigation

Key navigation commands are:

- `pwd` to print (the current) working directory.
- `cd` to change directory.

```
$ pwd
```

```
/Users/s.munzert/Documents/github/intro-to-data-science-26/lectures/04_command-line
```

You can use absolute paths, but it's better to use relative paths and invoke special symbols for a user's home folder (`~`), current directory (`.`), and parent directory (`..`) as needed.

```
$ cd examples ## Move into the "examples" sub-directory of this lecture directory.
```

```
$ cd ../../.. ## Now go back up two directories.
```

```
$ pwd
```

```
/Users/s.munzert/Documents/github/intro-to-data-science-26/lectures
```

Navigation (cont.)

Beware of directory names that contain spaces. Say you have a directory called "My Documents". (I'm looking at you, Windows.)

- Why won't `$ cd My Documents` work?
- Bash syntax is super pedantic about spaces and ordering. Here it thinks that "My" and "Documents" are separate arguments.
- How to deal with it:
 - Use quotation marks: `$ cd "My Documents"`.
 - Use Tab completion to automatically "escape" the space: `$ cd My\ Documents`.
 - Don't use spaces in file and folder names. Just don't.
 - I've developed the habit to name files and folders (a) always lowercase and (b) using dashes, as in `assignment-05`. It's a useful convention and frees up cognitive resources for more important decisions.

Fail.

```
$ cd My Documents
```

Use quotation.

```
$ cd "My Documents"
```

Use tab completion.

```
$ cd My\ Documents
```

Don't use spaces.

```
$ cd MyDocuments
```



Listing files and their properties

We're about to go into more depth about the `ls` (list) command. It shows the contents of the current (or given) directory:

```
$ ls
```

```
04-command-line.html
04-command-line.pdf
04-command-line.Rmd
examples
libs
pics
simons-touch.css
```

Now we list the contents of the `examples/` sub-directory with the `-lh` option ("long format", "human readable file size unit suffixes"; again, check out `man ls` for the details):

```
$ ls -lh examples
```

```
total 288
```

```
drwxr-xr-x@ 3 s.munzert  staff    96B Sep 23 22:08 ABC
drwxr-xr-x@ 3 s.munzert  staff    96B Sep 23 21:03 backup-snapshot
drwxr-xr-x@ 5 s.munzert  staff   160B Sep 23 21:54 batch-image-conversion
drwxr-xr-x@ 4 s.munzert  staff   128B Sep 23 21:03 bulk-rename
-rw-r--r--@ 1 s.munzert  staff   653B Sep 18 17:38 error.log
-rw-r--r--@ 1 s.munzert  staff    30B Sep 20 14:11 hello.R
-rwxr-xr-x@ 1 s.munzert  staff    34B Aug 17 15:01 hello.sh
drwxr-xr-x@ 5 s.munzert  staff   160B Sep 23 13:16 log-analysis
drwxr-xr-x@ 9 s.munzert  staff   288B Sep 23 22:08 meals
-rw-r--r--@ 1 s.munzert  staff    32B Aug 17 15:01 nursery.txt
-rw-r--r--@ 1 s.munzert  staff    38B Aug 17 15:01 nursery2.txt
drwxr-xr-x@ 5 s.munzert  staff   160B Sep 23 21:56 pdf-toolbox
-rwxr-xr-x@ 1 s.munzert  staff   153B Aug 17 15:01 reps.txt
-rw-r--r--@ 1 s.munzert  staff   117K Aug 17 15:01 sonnets.txt
```

Listing files and their properties (cont.)

What does this all mean? Let's focus on the top line.

```
drwxr-xr-x@ 3 s.munzert staff 96B Sep 20 13:42 ABC
```

- The first column denotes the object type:
 - `d` (directory or folder), `l` (link), or `-` (file)
- Next, we see the permissions associated with the object's three possible user types: 1) owner, 2) the owner's group, and 3) all other users.
 - Permissions reflect `r` (read), `w` (write), or `x` (execute) access.
 - `-` denotes missing permissions for a class of operations.
- The number of hard links to the object.
- We also see the identity of the object's owner and their group.
- Finally, we see some descriptive elements about the object:
 - Size, date and time of creation, and the object name.

Summary

- The `pwd` (print working directory) command shows the current working directory.
- The `ls` (list) command shows the contents of the current directory or a given directory.
- The `ls -l` command shows the contents of the current directory as list.
- The `cd` (change directory) changes the current working directory.
- You can run `cd` at any time to quickly go to your home directory.
- You can use the `cd -` command to go back to the last location.
- Absolute paths are paths which specify the exact location of a file or folder.
- Relative paths are paths which are relative to the current directory.
- The `.` special folder means 'this folder'.
- The `..` special folder means 'the parent folder'.
- The `~` special folder is the 'home directory'.
- The `$PWD` environment variable holds the current working directory.
- The `$HOME` environment variable holds the user's home directory.
- The `tree` command can show the files and folders in a given directory. (Install first on a Mac.)
- The `file` command can be used to ask the shell what it thinks a file is.

For a more detailed overview, see [here](#).

Managing your files

Managing your files

- The obvious next step after navigating the file system is managing files.
- There's a lot you can do with files, including downloading, unzipping, copying, moving, renaming and deleting.
- Again, doing this in a GUI is intuitive but usually scales badly.
- We'll learn how to do these operations at scale using the shell.
- Be careful when handling files in the shell though! Don't expect friendly reminders such as "Do you really want to delete this folder of pictures from your anniversary?"



Create: touch and mkdir

One of the most common shell tasks is object creation (files, directories, etc.).

We use `mkdir` to create directories. E.g., to create a new "testing" directory we do:

```
$ mkdir testing
```

We use `touch` to create (empty) files. If the file(s) already exist, `touch` changes a file's "Access", "Modify" and "Change" timestamps to the current time and date. To add some files to our new directory, we do:

```
$ touch testing/test1.txt testing/test2.txt testing/test3.txt
```

Check that it worked:

```
$ ls testing
```

```
test1.txt  
test2.txt  
test3.txt
```

Remove: rm and rmdir

Let's delete the objects that we just created. Start with one of the .txt files, by using `rm`.

- We could delete all the files at the same time, but you'll see why I want to keep some.

```
$ rm testing/test1.txt
```

The equivalent command for directories is `rmdir`.

```
$ rmdir testing
```

```
rmdir: testing: Directory not empty
```

Uh oh... It won't let us delete the directory while it still has files inside of it. The solution is to use the `rm` command again with the "recursive" (`-r` or `-R`) and "force" (`-f`) options.

- Excluding the `-f` option is safer, but will trigger a confirmation prompt for every file, which I'd rather avoid here.

```
$ rm -rf testing ## Success
```

Copy: cp

The syntax for copying is `$ cp object path/copyname`.

- If you don't provide a new name for the copied object, it will just take the old name.
- However, if there is already an object with the same name in the target destination, then you'll have to use `-f` to force an overwrite.

```
$ ## Create new "copies" sub-directory
$ mkdir examples/copies
$ ## Now copy across a file (with a new name)
$ cp examples/refs.txt examples/copies/refs-copy.txt
$ ## Show that we were successful
$ ls examples/copies
```

refs-copy.txt

You can use `cp` to copy directories, although you'll need the `-r` flag if you want to recursively copy over everything inside of it too:

```
$ cp -r examples/meals examples/copies
```

Move (and rename): mv

The syntax for moving is `$ mv object path/newobjectname`

```
$ ## Move the abc.txt file and show that it worked
$ mv examples/ABC/abc.txt examples
$ ls examples/ABC ## empty
```

```
$ ## Move it back again
$ mv examples/abc.txt examples/ABC
$ ls examples/ABC ## not empty
```

abc.txt

Note that "moving" an object within the same directory, but with specifying `newobjectname`, is effectively the same as renaming it.

```
$ ## Rename reps-copy to reps2 by "moving" it with a new name
$ mv examples/copies/reps-copy.txt examples/copies/reps2.txt
$ ls examples/copies
```

reps2.txt

Wildcards

Wildcards are special characters that can be used as a replacement for other characters. The two most important ones are:

1. Replace any number of characters with `*`.

- Convenient when you want to copy, move, or delete a whole class of files.

```
$ cp examples/*.sh examples/copies ## Copy any file with an .sh extension to "copies"
$ rm examples/copies/* ## Delete everything in the "copies" directory
```

2. Replace a single character with `?`

- Convenient when you want to discriminate between similarly named files.

```
$ ls examples/meals/??nday.csv
$ ls examples/meals/?onday.csv
```

```
examples/meals/monday.csv
examples/meals/sunday.csv
examples/meals/monday.csv
```

Find

The last command that I want to mention w.r.t. navigation is `find`.

This can be used to locate files and directories based on a variety of criteria; from pattern matching to object properties.

```
$ find examples -iname "monday.csv" ## will automatically do recursive, -iname makes search case-insensitive
```

```
examples/meals/monday.csv
```

```
$ find . -iname "*.txt" ## must use "." to indicate pwd
```

```
./examples/ABC/abc.txt
./examples/sonnets.txt
./examples/reps.txt
./examples/nursery.txt
./examples/bulk-rename/messy/Meeting Notes 3.txt
./examples/nursery2.txt
```

```
$ find . -size +2000k ## find files larger than 2000 KB
```

```
./04-command-line.pdf
./pics/bookshelf.gif
./pics/pipe-giphy.gif
./pics/pipe-giphy.gif
```

Summary

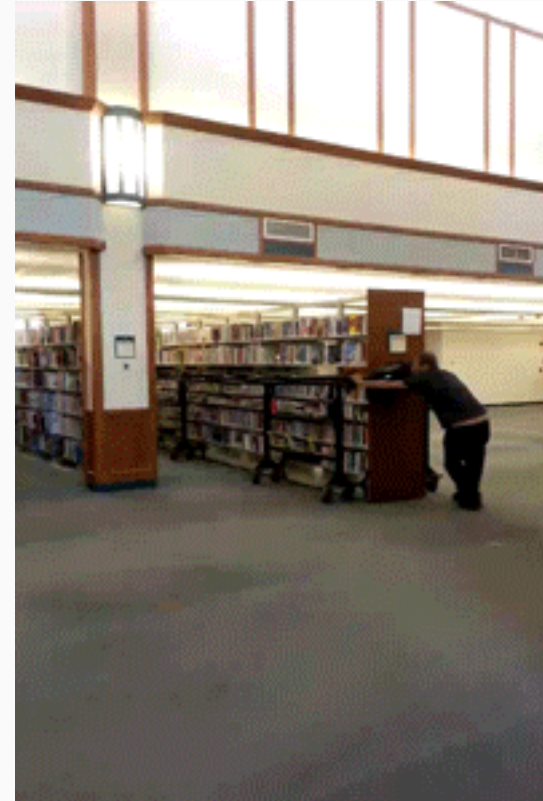
- The `rm` (remove) command can delete a file (they are gone forever, no recycle bin!).
- The `rm` command won't delete a folder which has files in it, unless you tell it to by adding the `-r` (recursive) flag.
- The `cp` (copy) command can copy a file.
- The `cp` can also be given wildcards like `*` to copy many files.
- The `mv` (move) command can move or rename a file.
- Renaming *en masse* (e.g. with wildcards or zsh's `zmv`) is a classic task to delegate to an AI assistant — but read the `mv` commands it proposes before you approve them.
- The `mkdir` command can create a folder - it can even create a whole tree of folders if you pass the `-p` (create parent directories) flag.
- The `find` command lets you find files based on specified criteria.
- The `cat` command (concatenated) can be used to write the contents of a file to the screen.
- We can pass multiple files to commands like `cat` if we use wildcards, such as `quotes/*`.
- The `wget` (web get) command can download a file from the web. (Install first on a Mac.)
- The `zip` / `unzip` commands can zip/unzip a file/folder for us.

For a more detailed overview, see [here](#).

Working with text files

Working with text files

- Data scientists spend a lot of time working with text, including scripts, Markdown documents, and delimited text files like CSVs.
- You will have the opportunity to learn more on the statistical analysis of text using NLP over the course of your studies.
- While Python and R are strong environments for text wrangling and analysis, it still makes sense to spend a few slides showing off some Bash shell capabilities for working with text files.
- We'll only scratch the surface, but hopefully you'll get an idea of how powerful the shell is in the text domain.



Counting text: wc

You can use the `wc` command to count:

1. The lines of text
2. The number of words
3. The number of characters

Let's demonstrate with a text file containing all of Shakespeare's Sonnets.¹

```
$ wc examples/sonnets.txt
```

```
3029    20701  119751 examples/sonnets.txt
```

(You couldn't tell here, but the character count is actually higher than we'd get if we (bothered) counting by hand, because `wc` counts the invisible newline character `"\n"`.)

¹ Courtesy of [Project Gutenberg](#).

Reading text

Read everything: cat

The simplest way to read in text is with the `cat` ("concatenate") command. Note that `cat` will read in *all* of the text. You can scroll back up in your shell window, but this can still be a pain.

Again, let's demonstrate using Shakespeare's Sonnets. (This will overflow the slide.) We also use the `-n` flag to show line numbers:

```
$ cat -n examples/sonnets.txt
```

```
1   The Project Gutenberg EBook of Shakespeare's Sonnets, by William Shakespeare
2
3   This eBook is for the use of anyone anywhere at no cost and with
4   almost no restrictions whatsoever.  You may copy it, give it away or
5   re-use it under the terms of the Project Gutenberg License included
6   with this eBook or online at www.gutenberg.org
7
8
9   Title: Shakespeare's Sonnets
10
11  Author: William Shakespeare
12
```

Reading text (cont.)

Scroll: more and less

The `more` and `less` commands provide extra functionality over `cat`. For example, they allow you to move through long text one page at a time. (While they look similar, `less` is more than `more`, **more or less...**)

- Try this yourself with `$ more examples/sonnets.txt`.
- You can move forward and back using the `f` and `b` keys, and quit by hitting `q`.

Preview: head and tail

The `head` and `tail` commands let you limit yourself to a preview of the text, down to a specified number of rows. (The default is 10 rows if you don't specify a number with the `-n` flag.)

```
$ head -n 3 examples/sonnets.txt ## First 3 rows
$ # head examples/sonnets.txt ## First 10 rows (default)
```

The Project Gutenberg EBook of Shakespeare's Sonnets, by William Shakespeare

This eBook is for the use of anyone anywhere at no cost and with

Reading text (cont.)

Preview: head and tail (cont.)

`tail` works very similarly to `head`, but starting from the bottom. For example, we can see the very last row of a file as follows:

```
$ tail -n 1 examples/sonnets.txt ## Last row
```

```
subscribe to our email newsletter to hear about new eBooks.
```

By using the `-n +N` option, we can specify that we want to preview all lines starting from row N *and after*, as in:

```
$ tail -n +3024 examples/sonnets.txt ## Show everything from line 3024
```

```
www.gutenberg.org
```

```
This Web site includes information about Project Gutenberg-tm,  
including how to make donations to the Project Gutenberg Literary  
Archive Foundation, how to help produce our new eBooks, and how to  
subscribe to our email newsletter to hear about new eBooks.
```

Find patterns: grep

To find patterns in text, we can use regular expression-type matching with `grep`.

For example, say we want to find the famous opening line to Shakespeare's **Sonnet 18**.

(We're going to include the `-n` ("number") flag to get the line that it occurs on.)

```
$ grep -n "Shall I compare thee" examples/sonnets.txt
```

```
336: Shall I compare thee to a summer's day?
```

By default, `grep` returns all matching patterns.

Check out what happens when we do the following:

```
$ grep -n "winter" examples/sonnets.txt
```

```
63:  When forty winters shall besiege thy brow,  
119: To hideous winter, and confounds him there;  
126:   But flowers distill'd, though they with winter  
132: Then let not winter's ragged hand deface,  
261: Against the stormy gusts of winter's day  
994:   Or call it winter, which being full of care,  
1679: How like a winter hath my absence been  
1692:   That leaves look pale, dreading the winter's  
1708:   Yet seem'd it winter still, and you away,  
1801: Such seems your beauty still. Three winters col
```

Find patterns: grep (cont.)

Note that `grep` can be used to identify patterns in a group of files (e.g. within a directory) too.

- This is particularly useful if you are trying to identify a file that contains, say, a function name.

Here's a simple example: Which days will I eat pasta this week?

- I'm using the `r` (recursive) and `l` (just list the files; don't print the output) flags.

```
$ grep -rl "pasta" examples/meals
```

```
examples/meals/monday.csv
```

Take a look at the `grep` man or cheat file for other useful examples and flags (e.g. `-i` for ignore case).

Manipulate text: sed

There are two main commands for manipulating text in the shell, namely `sed` and `awk`. Both of these are very powerful and flexible. We'll briefly look into `sed` for now. (Mac users, note that the MacOS `sed` works a bit differently; see [here](#).)

`sed` is the *stream editor* command. It takes input from a stream - which in many cases will simply be a file. It then performs operations on the text as it is read, and returns the output.

Example: Replace one text pattern with another.

```
$ cat examples/nursery.txt
```

```
Jack and Jill  
Went up the hill
```

```
$ sed 's/Jack/Bill/g' examples/nursery.txt  
$ cat examples/nursery.txt
```

```
Bill and Jill  
Went up the hill  
Jack and Jill  
Went up the hill
```

Let's look at the expression `s/Jack/Bill/g` in detail:

- The `s` indicates that we are going to run the substitute function, which is used to replace text.
- The `/` indicates the start of the pattern we are searching for - `Bill` in this case.
- The second `/` indicates the start of the replacement we will make when the pattern is found.
- The final `/` indicates the end of the replacement - we can also optionally put flags after this slash. Here, `g` ensures global replacement (not just replacement of the first match).

Summary

- `head` will show the first ten lines of a file.
- `head -n 30` will show the first thirty lines of a file, using the `-n` flag to specify the number of lines.
- `tail` will show the final ten lines of a file.
- `tail -n 3` uses the `-n` flag to specify three lines only.
- `tr 'a' 'b'` is the translate characters command, which turns one set of characters into another.
- `cut` can be used to extract parts of a line of text.
- `cut -d',' -f 3` shows how the `-d` or delimiter flag is used to specify the delimiter to cut on and how the `-f` or field flag specifies which of the fields the text has been cut into is printed.
- `cut -c 2-4` uses the `-c` or characters flag to specify that we are extracting a subset of characters in the line, in this case characters two to four.
- `rev` reverses text - by reversing, cutting and then re-reversing you can quickly extract text from the end of a line.
- `sort` sorts the incoming text alphabetically. The `-r` flag for `sort` reverses the sort order.
- The `uniq` command removes duplicate lines - but only when they are next to each other, so you'll often use it in combination with `sort`.
- Your pager, for example the `less` program can be useful when inspecting the output of your text transformation commands.

Also, make sure to master regular expressions!

Good starting points are:

- [This base R regex intro](#)
- [This intro from R4DS](#)
- [This vignette from the stringr package](#)
- And, of course, the class (or, more likely, the labs) next week!



Redirects, pipes, and loops

Redirects, pipes, and loops

- You have learned about pipes in R already.
- Understanding the concept of pipelines in the shell, as well as how input and output work for command line programs is critical to be able to use the shell effectively.
- Think again of the Unix philosophy of "doing one thing, but doing it well" and combining multiple of these modules.
- Also, often you'll want to dump output in a file as part of your workflow.
- Let's learn how all this works.



Redirect: >

You can send output from the shell to a file using the redirect operator `>`.

For example, let's print a message to the shell using the `echo` command.

```
$ echo "At first, I was afraid, I was petrified"
```

```
At first, I was afraid, I was petrified
```

If you wanted to save this output to a file, you need simply redirect it to the filename of choice.

```
$ echo "At first, I was afraid, I was petrified" > survive.txt  
$ find survive.txt ## Show that it now exists
```

```
survive.txt
```

Redirect: > (cont.)

If you want to *append* text to an existing file, then you should use `>>`.

- Using `>` will try to overwrite the existing file contents.

```
$ echo "'Kept thinking I could never live without you by my side'" >> survive.txt
$ cat survive.txt
```

```
At first, I was afraid, I was petrified
'Kept thinking I could never live without you by my side
```

An example use case is when adding rules to your .gitignore, e.g. `$ echo "*.csv" >> .gitignore.`

Pipes: |

The pipe operator `|` is one of the coolest features in Bash.

- It lets you send (i.e. "pipe") intermediate output to another command.
- In other words, it allows us to chain together a sequence of simple operations and thereby implement a more complex operation.

Here's a simple example:

```
$ cat -n examples/sonnets.txt | head -n100 | tail -n10
```

```
91      Despite of wrinkles this thy golden time.  
92          But if thou live, remember'd not to be,  
93          Die single and thine image dies with thee.  
94  
95      IV  
96  
97      Unthrifty loveliness, why dost thou spend  
98      Upon thy self thy beauty's legacy?  
99      Nature's bequest gives nothing, but doth lend,  
100     And being frank she lends to those are free:
```

Exit codes and command chaining

Every command reports back whether it succeeded: When it finishes, it returns an **exit code** — `0` means success, anything else means failure. The special variable `$?` stores the exit code of the last command.

```
$ ls examples/hello.sh
$ echo $? ## 0 = success
```

```
examples/hello.sh
0
```

```
$ ls examples/no-such-file.txt 2> /dev/null
$ echo $? ## non-zero = failure
```

```
1
```

Exit codes power the **conditional chaining operators**: `command1 && command2` runs `command2` only if `command1` *succeeded*; `command1 || command2` runs `command2` only if `command1` *failed*.

```
$ mkdir -p examples/tmp && echo "Folder exists, safe to continue."
$ ls examples/no-such-file.txt 2> /dev/null || echo "Fallback: file not found."
```

```
Folder exists, safe to continue.
Fallback: file not found.
```

Why care? Because all-or-nothing chains like `cd project && git pull && Rscript analysis.R` are everywhere — not least in the commands that AI coding agents propose to run on your machine. Be able to read them before you hit "approve".

Iteration with *for* loops

Sometimes you want to loop an operation over certain parameters. *for* loops in Bash/Z shell work similarly to other programming languages that you are probably familiar with.

The basic syntax is:

```
for i in LIST
do
  OPERATION $i ## the $ sign indicates a variable in bash
done
```

We can also condense things into a single line by using `;` appropriately.

```
for i in LIST; do OPERATION $i; done
```

Note: Using `;` isn't limited to *for* loops. Semicolons are a standard way to denote line endings in Bash/Z shell.

Example 1: Print a sequence of numbers

To help make things concrete, here's a simple *for* loop in action.

```
$ for i in 1 2 3 4 5; do echo $i; done
```

```
1  
2  
3  
4  
5
```

FWIW, we can use bash's brace expansion (`{1..n}`) to save us from having to write out a long sequence of numbers.

```
$ for i in {1..5}; do echo $i; done
```

```
1  
2  
3  
4  
5
```

Example 2: Combine CSVs

Here's a more realistic *for* loop use-case: Combining (i.e. concatenating) multiple CSVs.

Say we want to combine all the "daily" files in the `examples/meals` directory into a single CSV, which I'll call `mealplan.csv`. Here's one attempt that incorporates various bash commands and tricks that we've learned so far. The basic idea is:

1. Create a new (empty) CSV
2. Then, loop over the relevant input files, appending their contents to our new CSV

```
## create an empty CSV
$ touch examples/meals/mealplan.csv
## loop over the input files and append their contents to our new CSV
$ for i in $(ls examples/meals/*day.csv)
> do
>   cat $i >> examples/meals/mealplan.csv
> done
```

Did it work? (See next slide.)

Example 2: Combine CSVs (cont.)

```
$ cat examples/meals/mealplan.csv
```

```
day,breakfast,lunch,dinner
friday,pancakes,ramen,stew
day,breakfast,lunch,dinner
monday,muesli,sandwich,pasta
day,breakfast,lunch,dinner
saturday,muesli,sandwich,pad thai
day,breakfast,lunch,dinner
sunday,muesli,roast,leftovers
day,breakfast,lunch,dinner
thursday,muesli,salad,tacos
day,breakfast,lunch,dinner
tuesday,muesli,soup,roast
day,breakfast,lunch,dinner
wednesday,muesli,sandwich,pizza
```

Hmmm. Sort of, but we need to get rid of the repeating header.

Can you think of a way? (*Hint:* `tail` and `head ...`)

Example 2: Combine CSVs (cont.)

Let's try again. Here's our adapted gameplan:

- First, delete the old file. Then, create the new file by grabbing the header (i.e. top line) from any of the input files and redirecting it. No need for `touch` this time.
- Next, loop over all the input files as before, but this time only append everything *after* the top line.

```
$ rm -f examples/meals/mealplan.csv ## delete old file
## create a new CSV by redirecting the top line of any file
$ head -1 examples/meals/monday.csv > examples/meals/mealplan.csv
## loop over the input files, appending everything after the top line
$ for i in $(ls examples/meals/*day.csv)
> do
>   tail -n +2 $i >> examples/meals/mealplan.csv
> done
```

It worked! (Check the output with `cat examples/meals/mealplan.csv`.) Why do this in the shell rather than in R? Efficiency: the files never have to be held in memory (i.e. RAM) simultaneously. This doesn't matter here, but makes a dramatic difference with lots of files (or a few really big ones).

Scripting

Scripting

Writing code **interactively** in the shell makes a lot of sense when you are exploring data, file structures, etc.

However, it's also possible (and often desirable) to **write reproducible shell scripts** that combine a sequence of commands.

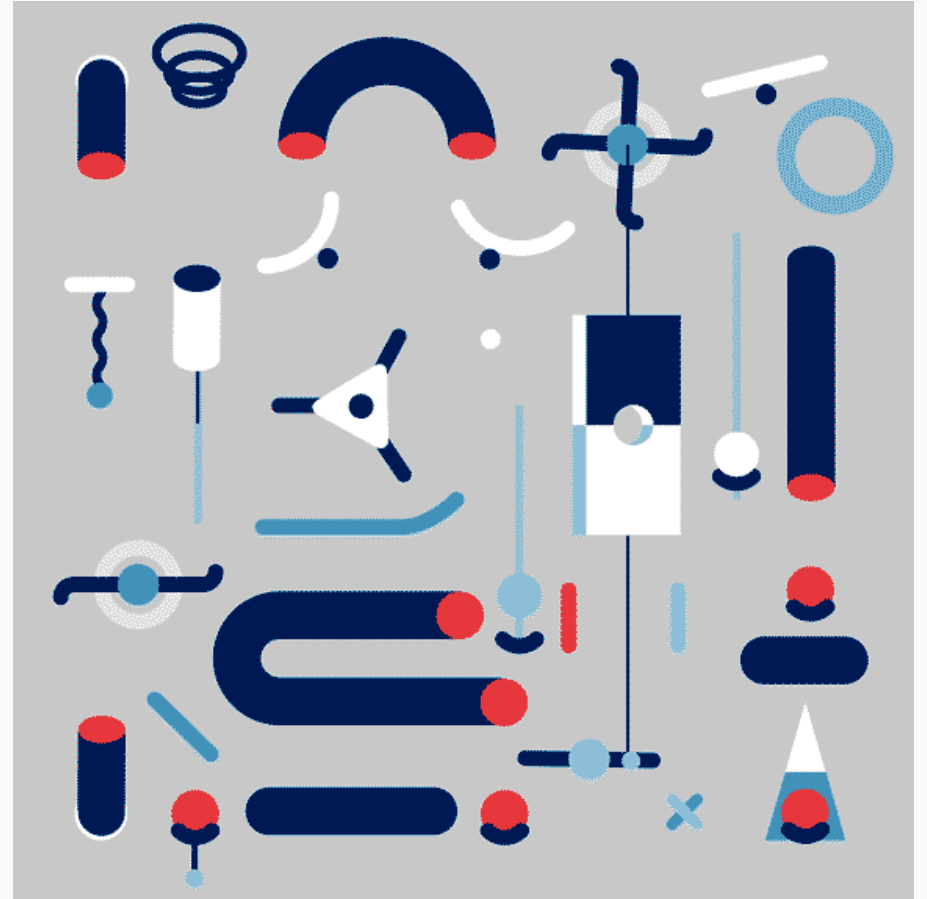
These scripts are demarcated by their `.sh` file extension.

Let's look at the contents of a short shell script, `hello.sh`, that is included in the examples folder:

```
$ cat examples/hello.sh
```

```
#!/bin/sh  
echo "\nHello World!\n"
```

What does this script do?



Hello World! (cont.)

```
#!/bin/sh  
echo "\nHello World!\n"
```

- `#!/bin/sh` is a **shebang**, indicating which program to run the command with (here: any Bash-compatible shell). However, it is typically ignored (note that it begins with the hash comment character.)
- `echo "\nHello World!\n"` is the actual command that we want to run.

To run this simple script, you can just type in the file name and press enter.

```
$ examples/hello.sh  
$ # bash examples/hello.sh ## Also works
```

Hello World!

It's important to realize that we aren't limited to running shell scripts in the shell. The exact same principles carry over to other programs and files.

The most relevant case for this class is the `Rscript` command for (you guessed it) executing R scripts and expressions. For example:

```
$ Rscript -e "cat('Hello World, from R!')"
```

```
Hello World, from R!
```

Of course, the more typical `Rscript` use case is to execute full length R scripts.

```
$ Rscript examples/hello.R
```

```
Hello World, from R!
```

Editing and writing scripts in the shell

Say you want to edit the `hello.sh` script. We have already seen how to append text lines to a file. But when it comes to more complicated editing work, you're better off using a command-line editor:

- An easy starting point is **nano**. (**Windows users**: recent versions of Git Bash ship with nano; if yours doesn't, see [here](#).)
- Another popular (and nerdy) option is **vim**. Extremely powerful, but a steep learning curve (I am told).
- More options [here](#).

A key advantage of command-line editors is that using them fits the command-line workflow - the keyboard is the only hardware input device.

But it's also absolutely legit to open (and modify) `.sh` scripts with your ordinary text editor. It will break the "flow" though.

With `nano`, open the script by typing `$ nano examples/hello.sh`.

- Note that the functionality is more limited than a normal text editor.
- Once you are finished editing, hit `ctrl + x`, then `y` and `enter` to exit.
- Finally, run the edited version of the script.

The shell in the age of AI agents

AI agents live in the shell

- The most capable AI coding agents — Claude Code, Codex CLI, Gemini CLI, aider — are terminal applications.
- More importantly, they **act through the shell**: they list files, `grep` through code, run `git`, install packages, and execute scripts — with the same commands you learned today.
- Before running a consequential command, an agent shows it to you and asks for permission. **Reading shell has become a supervision skill.**
- Agents will also happily *write* shell commands for you. For you, that means producing clever one-liners matters less; reading commands, understanding the environment they run in, and cleaning up when something goes wrong matters more.

Before you hit "approve", ask yourself:

1. What does the command touch? Watch out for paths and wildcards (`rm -rf *` in the wrong directory is a catastrophe).
2. Is it destructive or irreversible? `rm` deletes forever, `mv` can overwrite, `>` truncates a file.
3. Where will it run? An agent's working directory may not be where you think it is.
4. What does it install or download, and from where? Do you trust the source?

If you can't answer these, don't approve — ask the agent to explain first. (It's good at that, too.)

Your environment: variables

Programs you start inherit **environment variables** — named settings that live in your shell session. Some are set for you:

```
$ echo $HOME
$ echo $SHELL
```

```
/Users/s.munzert
/bin/zsh
```

Set your own with `export`:

```
$ export ANTHROPIC_API_KEY="sk-ant-..." ## now every program started from this shell can read it
```

- By convention, names are in `UPPERCASE`. Inspect all currently set variables with `printenv`.
- An `export` ed variable lasts only for the current session. To make it permanent, add the `export` line to your shell's configuration file (`~/.zshrc` for zsh, `~/.bashrc` for bash), which runs every time a new shell starts.
- **Why this matters now:** Environment variables are *the* standard way to configure command-line tools — including AI agents, which read their API keys from variables like `ANTHROPIC_API_KEY`. Never hardcode keys in scripts you share!

Your environment: the PATH

How does the shell know what `ls` or `Rscript` is? It searches the directories listed in the `PATH` variable, in order, for an executable with that name.

```
$ echo $PATH | tr ':' '\n' | head -4 ## one directory per line (note the pipes!)
```

```
/usr/local/bin  
/System/Cryptexes/App/usr/bin  
/usr/bin  
/bin
```

`which` tells you *where* a command was found:

```
$ which R
```

```
/usr/local/bin/R
```

- The infamous `command not found` error simply means: no executable with that name was found in any `PATH` directory. The fix is usually to install the tool — or to add its directory to `PATH` in your `~/.zshrc`.
- When you install a CLI tool — say, `npm install -g @anthropic-ai/claude-code` — the installer drops an executable into a `PATH` directory. That's the whole magic behind a new command "existing".

Processes: what is my computer doing?

Every running program is a **process** with a unique process ID (PID).

```
$ ps ## show the processes running in this shell session
```

PID	TTY	TIME	CMD
81321	ttys005	0:00.13	-zsh
84102	ttys005	0:04.71	Rscript analysis.R

- `ps -e` shows *all* processes on the machine; `top` (or the friendlier `htop`) shows them live, ordered by resource use.
- Stop the process running in the foreground with `Ctrl + C`.
- Stop any process with `kill <PID>` (or, as a last resort, `kill -9 <PID>`).
- Append `&` to run a command in the **background** and keep working: `Rscript slow-job.R &`.

Why this matters now: AI agents love to start web servers and long-running jobs. When your agent leaves a runaway process behind (or your laptop fan takes off), `ps`, `top`, and `kill` are the rescue squad.

The Unix philosophy, vindicated

Remember: *Do one thing and do it well* — small tools connected by text streams. This 50-year-old design is a perfect match for LLMs, which are also text in, text out. Your new AI tools compose with the old Unix ones:

```
$ cat examples/error.log | claude -p "In one sentence: what went wrong in this log?"
```

The scraper job failed on Sept 15 because `spiegel.de` rate-limited the request (HTTP 429), so no headlines were parsed and the script halted.

```
$ git diff | claude -p "Write a one-line commit message for these changes."
```

- Used this way, the LLM is just another command-line tool: it reads from a pipe and writes to standard output. So you can put it *in the middle* of a pipeline, redirect its output to a file, or call it in a loop.
- The skill that ties it all together is the one you learned today: knowing what the individual pieces do, and how to plug them together.

Ctrl+d

Things we didn't cover today

We covered a *lot* of ground today. I hope that I've given you a sense of how the shell works and how powerful it is.

My main goal has been to "demystify" the shell, so that you aren't intimidated when we use shell commands later on.

At the same time, there's loads that we didn't cover.

- User roles and file permissions, SSH, job scheduling with cron, GNU parallel, etc.
- Automation; see [here](#), [here](#), and [here](#) are great places to start learning about automation on your own.

If you want to dig deeper, check out

- [Data Science at the Command Line](#) (Jeroen Janssens)
- [Effective Shell](#) (Dave Kerr)
- [Terminal Trove](#) — a browsable, curated catalog of modern command-line tools, updated constantly.
- [awesome-cli-apps](#) and [awesome-shell](#) — the canonical community-curated lists on GitHub.
- [explainshell.com](#) — paste any command and every flag gets annotated from the man pages. Perfect for decoding commands you found online — or commands an AI agent wants to run on your machine.
- [Using AWK and R to parse 25tb](#) (Nick Strayer)

A shell toolbox for data scientists

The built-ins are just the start — a rich ecosystem of installable CLI tools is especially useful for data work (all installable via your package manager, e.g. `brew install duckdb jq` on macOS):

Tool	What it does
<code>jq</code>	filter and reshape JSON — the <code>dplyr</code> of API responses
<code>curl</code>	talk to any web API
<code>csvkit</code> / <code>qsv</code>	CSV toolkits: instant summary stats, delimiter-aware cut/grep
<code>VisiData</code>	interactive spreadsheet in the terminal — pivot a 2 GB CSV
<code>DuckDB</code>	run SQL directly on CSV/Parquet files — plays nicely with R
<code>tmux</code>	keep shell sessions alive on remote servers
<code>GNU parallel</code>	parallelize any loop over files or arguments
<code>pandoc</code> & friends	convert documents (pandoc), PDFs (poppler), images (ImageMagick), media (ffmpeg)
<code>llm</code> / <code>claude</code>	LLMs as pipeable command-line tools